

claude-windows / 75d3d121-f240-4293-afa5-b436a379ad31

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\75d3d121-f240-4293-afa5-b436a379ad31\subagents\workflows\wf_5f13fc79-bad\agent-ad9dae007534b8ef3.jsonl`  
- SHA-256: `56e373991d27d9be1424eb19314d54598e9db230037326850eb98b679cca39d9`  
- Source modified: `2026-07-08T06:18:16+00:00`  
- Imported at: `2026-07-08T15:59:28+00:00`  
- project: `wf_5f13fc79-bad`  
- session_id: `75d3d121-f240-4293-afa5-b436a379ad31`
```

Transcript

1. user (2026-07-08T06:16:23.552Z)

CONTEXT: This is GitHub issue #521 (Khelsutra/badminton-highlight-indexer), MERGED as PR #525. It makes pipeline phase?machine placement config-driven and dispatches the reel compile/stitch to an L4 Cloud Run worker (NVENC), mirroring the existing detection dispatch.

The full unified diff is at: undefined (read it first).

The full post-change source is under: undefined/ (read whole files there for context ? e.g. undefined/backend/orchestration.py, undefined/backend/worker/__main__.py, undefined/backend/compute/cloud_run.py, undefined/backend/config/placement.py, undefined/backend/config/models.py, undefined/backend/pipeline/stitcher/compiler.py).

Key design invariants that MUST hold (a violation is a real bug):

- DEFAULT-OFF: with no phase_placement config and compute_target unset, behaviour is byte-identical to before #521 (validation+compile on control-plane, detect in-process).
- The cloud compile path must return the SAME {status, filepath, download_url, video_id} shape the SPA already polls for; a stitch failure must be a clean failed-job, never a crash.
- The worker 'compile' subcommand reads a bucket-staged spec (resolved time-pairs + stitching config + source uri) and needs NO control-plane DB. It reuses orchestration.__stitch_reel (shared core).
- Encoder: all clips in one reel must share ONE codec so the concat stays -c copy. NVENC is probed once and falls back to libx264 if unavailable (CI/non-GPU) ? the fallback must be all-or-nothing, never per-clip (which would break the concat).
- The config guard rejects validation=cloud_run_l4 without a cloud segment (nobody-validates footgun).
- Marker prefixes: compile uses _compile/, detect uses _dispatch/ ? must not collide.

Only report DEFECTS in the changed code (logic bugs, broken invariants, edge cases that produce a wrong result / crash / silent data loss, or missing test coverage of a real failure mode). Do NOT report style, naming, or speculative concerns. Cite exact file:line from undefined.

A reviewer claims this DEFECT in the merged #521 code. Try HARD to REFUTE it by reading the actual code at undefined. Default to is_real=false if you cannot concretely confirm the failure. Skeptic #1.

CLAIM (low/correctness) backend/pipeline/stitcher/compiler.py:31

summary: The NVENC availability probe does not use the same encoder args as the real per-clip encode, so it can report an encoder "runnable" that then fails on every actual clip with no fallback.

failure scenario: `_probe_encoder_available` (compiler.py:31) test-encodes with only `'-c:v h264_nvenc'` and no preset/quality flags, whereas `_select_encoder` (compiler.py:294-297) runs every real clip with `'-c:v h264_nvenc -preset p4 -cq 23 -pix_fmt yuv420p'`. On an ffmpeg build where basic NVENC initialises but the p1-p7 preset naming (`'-preset p4'`) is unsupported (older nvenc preset scheme), the probe returns True, the encoder is committed once for the whole run, then every `_trim_one_segment` encode fails; `temp_clips` ends up empty and `compile_highlights` raises `RuntimeError` -> the reel fails entirely, when a probe that matched the real command would have fallen back to `libx264` and produced a reel. The probe's stated contract (`'a real test encode ... distinguishes usable from will-fail-at-runtime'`) is not met. Version-gated (modern ffmpeg on the L4 image is unaffected), hence low confidence.

2. user (2026-07-08T06:16:32.573Z)

```
1     import logging
2     import os
3     import subprocess
4     import tempfile
5     from pathlib import Path
6     from typing import Any, Dict, List, Optional, Tuple, Union
7
8     from backend.utils.ffmpeg import ( # P2a: single ffmpeg/ffprobe resolver
9         ffmpeg_bin,
10        ffmpeg_timeout_sec,
11        ffprobe_bin,
12        ffprobe_timeout_sec,
13        timeout_label,
14    )
15
16    logger = logging.getLogger(__name__)
17
18    # Bundled, commercially-clean default watermark font (Apache-2.0). Shipped so the
overlay
19    # renders out-of-the-box without a system fontconfig setup (a common gap on Windows,
where
20    # `font='Sans'` fails). Overridable via stitching.watermark.font_path. This module lives
at
21    # backend/pipeline/stitcher/, so parents[2] == backend/.
22    _BUNDLED_FONT = (
23        Path(__file__).resolve().parents[2] / "assets" / "fonts" / "RobotoSlab-Regular.ttf"
24    )
25
26    #: Per-encoder availability memo (#521): probing ffmpeg is a subprocess, so a run's
first clip pays
27    #: it and the rest reuse the verdict. Keyed by encoder name; cleared only across
processes.
28    _ENCODER_AVAILABLE: Dict[str, bool] = {}
29
30
31    def _probe_encoder_available(encoder: str) -> bool:
32        """Can this ffmpeg build actually RUN ``encoder`` (not merely list it)? Runs a tiny
1-frame
33        lavfi test encode to null. NVENC is often *compiled in* on boxes without an NVIDIA
GPU, so a
34        real test encode ? not ``-encoders`` grep ? is what distinguishes "usable" from
"will fail at
35        runtime". Memoized. Any error / timeout ? unavailable (caller falls back to
libx264)."""
36        if encoder in _ENCODER_AVAILABLE:
37            return _ENCODER_AVAILABLE[encoder]
38        ok = False
39        # The probe is a tiny 1-frame encode ? bound it to a short timeout (? the configured
```

```

ffmpeg
40     # budget, or 30 s when unbounded) so a wedged encoder can't hang the whole stitch.
41     configured = ffmpeg_timeout_sec()
42     probe_timeout = min(30.0, float(configured)) if configured else 30.0
43     try:
44         proc = subprocess.run(
45             [
46                 ffmpeg_bin(),
47                 "-hide_banner",
48                 "-f",
49                 "lavfi",
50                 "-i",
51                 "color=c=black:s=64x64:d=0.1",
52                 "-frames:v",
53                 "1",
54                 "-c:v",
55                 encoder,
56                 "-f",
57                 "null",
58                 "-",
59             ],
60             capture_output=True,
61             timeout=probe_timeout,
62         )
63         ok = proc.returncode == 0
64     except Exception: # noqa: BLE001 ? a probe failure/timeout just means "not usable ?
        fall back"
65         ok = False
66         _ENCODER_AVAILABLE[encoder] = ok
67     return ok
68
69
70     class VideoCompiler:
71         def __init__(
72             self,
73             output_dir: str,
74             end_padding: float = 1.0,
75             begin_padding: float = 0.5,
76             watermark: Optional[Any] = None,
77             encoder: str = "libx264",
78             encode_profile: Optional[Any] = None,
79         ):
80             self.output_dir = output_dir
81             self.end_padding = end_padding
82             self.begin_padding = begin_padding
83             # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
84             # normalized to a dict (or None) so this module stays config-package-agnostic.
85             self.watermark = self._normalize_watermark(watermark)
86             # Video encoder for the per-clip trim re-encode (#521). "libx264" (CPU) is the
87             # works everywhere; "h264_nvenc"/"hevc_nvenc" use the L4's HARDWARE encoder for
88             # re-encode the watermark forces. The requested encoder is PROBED once per run
89             # (_select_encoder) and falls back to libx264 if this ffmpeg build can't run it.
90             self.encoder = (encoder or "libx264").strip() or "libx264"
91             self.encode_profile = self._normalize_encode_profile(encode_profile)
92             os.makedirs(output_dir, exist_ok=True)
93
94         @staticmethod
95         def _normalize_watermark(watermark: Optional[Any]) -> Optional[Dict[str, Any]]:
96             if watermark is None:
97                 return None
98             if hasattr(watermark, "model_dump"):
99                 return watermark.model_dump()
100             if isinstance(watermark, dict):

```

```

101         return dict(watermark)
102     return None
103
104     @staticmethod
105     def _normalize_encode_profile(profile: Optional[Any]) -> Dict[str, Any]:
106         if profile is None:
107             raw: Dict[str, Any] = {}
108         elif hasattr(profile, "model_dump"):
109             raw = profile.model_dump()
110         elif isinstance(profile, dict):
111             raw = dict(profile)
112         else:
113             raw = {}
114         return {
115             "max_width": int(raw.get("max_width") or 0),
116             "max_height": int(raw.get("max_height") or 0),
117             "crf": int(raw.get("crf", 22)),
118             "audio_bitrate": str(raw.get("audio_bitrate") or "128k"),
119             "video_preset": str(raw.get("video_preset") or "superfast"),
120             "pix_fmt": str(raw.get("pix_fmt") or ""),
121         }
122
123     @staticmethod
124     def _ff_quote(path: str) -> str:
125         """Make a filesystem path safe to embed inside single quotes in an ffmpeg
126         filtergraph operand (drawtext fontfile/textfile, movie= source): forward
127         slashes (Windows-friendly), escaped single quotes, AND escaped colons.
128
129         The colon MUST be escaped even inside single quotes: ffmpeg's filtergraph
130         option parser treats ':' as the option separator, so a Windows drive-letter
131         path like ``C:/dir/font.ttf`` otherwise truncates the operand at ``C`` and
132         the encode fails with "Invalid argument". This silently disabled the
133         on-by-default bundled-font watermark (and absolute logo paths) on Windows."""
134         return path.replace("\\", "/").replace("'", r"\'").replace(":", r"\:")
135
136     @staticmethod
137     def _parse_time(val: Union[int, float, str]) -> float:
138         """Normalizes a timestamp value to float seconds.
139         Handles: float/int (pass-through), plain numeric strings ("12.4"),
140         and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
141         This mirrors the parse_time logic used in gemini.py to ensure
142         the stitcher can always consume whatever format the indexer produced."""
143         if isinstance(val, (int, float)):
144             return float(val)
145         if isinstance(val, str):
146             val = val.strip()
147             if ":" in val:
148                 # Handle MM:SS, HH:MM:SS, etc.
149                 parts = val.split(":")
150                 parts.reverse()
151                 total = 0.0
152                 for i, p in enumerate(parts):
153                     try:
154                         total += float(p) * (60**i)
155                     except ValueError:
156                         pass
157                 return total
158             try:
159                 return float(val)
160             except ValueError:
161                 logger.warning(
162                     f"Could not parse timestamp value '{val}' as a number. Defaulting to
0.0."
163                 )
164                 return 0.0

```

```

165         logger.warning(
166             f"Unexpected timestamp type {type(val).__name__} for value '{val}'.
Defaulting to 0.0."
167         )
168         return 0.0
169
170     @staticmethod
171     def _merge_overlapping(
172         segments: List[Tuple[float, float]],
173     ) -> List[Tuple[float, float]]:
174         """Normalize, sort, and merge overlapping/duplicate (start, end) ranges.
175
176         A highlight reel must never replay the same footage. The same rally can land in
177         the segment list more than once ? e.g. two detector runs accumulated for one
178         video (add_video is an UPSERT, so re-running without a prune leaves both sets),
179         or
180         chunk-overlap duplicates near boundaries. Stitching those verbatim plays the
181         rally
182         twice. Merging any range that overlaps or abuts the previous one collapses true
183         duplicates/overlaps while preserving genuinely distinct (gapped) rallies.
184         Degenerate rows (end <= start) are dropped. Timestamps may be float/int/str.
185         """
186         parsed = []
187         for raw_s, raw_e in segments:
188             s = VideoCompiler._parse_time(raw_s)
189             e = VideoCompiler._parse_time(raw_e)
190             if e > s:
191                 parsed.append((s, e))
192         parsed.sort()
193         merged: List[Tuple[float, float]] = []
194         for s, e in parsed:
195             if merged and s <= merged[-1][1]: # overlaps/abuts previous -> extend it
196                 merged[-1] = (merged[-1][0], max(merged[-1][1], e))
197             else:
198                 merged.append((s, e))
199         return merged
200
201     def _get_video_duration(self, video_path: str) -> float:
202         """Probes the video file to get its total duration in seconds.
203         Returns 0.0 if it cannot be determined."""
204         try:
205             timeout = ffprobe_timeout_sec()
206             result = subprocess.run(
207                 [
208                     ffprobe_bin(),
209                     "-v",
210                     "error",
211                     "-show_entries",
212                     "format=duration",
213                     "-of",
214                     "default=noprint_wrappers=1:nokey=1",
215                     video_path,
216                 ],
217                 capture_output=True,
218                 text=True,
219                 check=True,
220                 timeout=timeout,
221             )
222             return float(result.stdout.strip())
223         except subprocess.TimeoutExpired:
224             logger.warning(
225                 "Could not determine video duration via ffprobe: timed out after %s",
226                 timeout_label(ffprobe_timeout_sec()),
227             )
228             return 0.0

```

```

227         except Exception as e:
228             logger.warning(f"Could not determine video duration via ffprobe: {e}")
229             return 0.0
230
231     def _watermark_filter(self, tmp_dir: str) -> Optional[str]:
232         """Build the ffmpeg -vf filtergraph that burns the configured watermark (text
233         and/or logo PNG) into a clip during the per-clip re-encode. Returns None when
234         watermarking is disabled or has nothing to draw. The concat stage stays -c copy
235         because the mark is already baked into every clip identically."""
236         wm = self.watermark
237         if not wm or not wm.get("enabled"):
238             return None
239
240         text = (wm.get("text") or "").strip()
241         logo_path = (wm.get("logo_path") or "").strip()
242         if logo_path and not os.path.exists(logo_path):
243             logger.warning(
244                 f"[watermark] logo not found: {logo_path} ? skipping logo overlay."
245             )
246             logo_path = ""
247         if not text and not logo_path:
248             return None
249
250         margin = int(wm.get("margin", 24))
251         opacity = float(wm.get("opacity", 0.85))
252         position = str(wm.get("position", "bottom-right"))
253         # drawtext uses lowercase w/h (main) + tw/th (text); overlay uses W/H (main) +
w/h (logo).
254         dt_xy = {
255             "bottom-right": f"x=w-tw-{margin}:y=h-th-{margin}",
256             "bottom-left": f"x={margin}:y=h-th-{margin}",
257             "top-right": f"x=w-tw-{margin}:y={margin}",
258             "top-left": f"x={margin}:y={margin}",
259         }.get(position, f"x=w-tw-{margin}:y=h-th-{margin}")
260         ov_xy = {
261             "bottom-right": f"W-w-{margin}:H-h-{margin}",
262             "bottom-left": f"{margin}:H-h-{margin}",
263             "top-right": f"W-w-{margin}:{margin}",
264             "top-left": f"{margin}:{margin}",
265         }.get(position, f"W-w-{margin}:H-h-{margin}")
266
267         drawtext = ""
268         if text:
269             ratio = float(wm.get("font_size_ratio") or 0.035)
270             divisor = max(1, round(1.0 / ratio))
271             # textfile= sidesteps the notorious drawtext text= escaping for arbitrary
strings.
272             tf_path = os.path.join(tmp_dir, "watermark.txt")
273             with open(tf_path, "w", encoding="utf-8") as fh:
274                 fh.write(text)
275             # Font precedence: explicit font_path ? bundled Apache-2.0 default (renders
276             # without fontconfig) ? fontconfig family (last-resort, environment-
dependent).
277             font_path = (wm.get("font_path") or "").strip()
278             if not font_path and _BUNDLED_FONT.exists():
279                 font_path = str(_BUNDLED_FONT)
280             if font_path:
281                 font_opt = f"fontfile='{self._ff_quote(font_path)}'"
282             else:
283                 font_opt = f"font='{wm.get('font', 'Sans')}'"
284             box = ""
285             if wm.get("box", True):
286                 box = f":box=1:boxcolor=black@{min(opacity, 0.5):.2f}:boxborderw=10"
287             drawtext = (
288                 f"drawtext=textfile='{self._ff_quote(tf_path)}':{font_opt}"

```

```

289         f":fontcolor=white@{opacity:.2f}:fontsize=h/{divisor}:{dt_xy}{box}"
290     )
291
292     if logo_path and drawtext:
293         return
294     f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy},{drawtext}"
295     if logo_path:
296         return f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy}"
297     return drawtext
298
299     def _scale_filter(self) -> Optional[str]:
300         max_width = int(self.encode_profile.get("max_width") or 0)
301         max_height = int(self.encode_profile.get("max_height") or 0)
302         filters = []
303         if max_width > 0 and max_height > 0:
304             filters.append(
305                 f"scale=w='min({max_width},iw)':h='min({max_height},ih)'"
306                 ":force_original_aspect_ratio=decrease:force_divisible_by=2"
307             )
308         pix_fmt = str(self.encode_profile.get("pix_fmt") or "").strip()
309         if pix_fmt:
310             filters.append(f"format={pix_fmt}")
311         return ",".join(filters) if filters else None
312
313     @staticmethod
314     def _combine_video_filters(
315         scale_filter: Optional[str], watermark_filter: Optional[str]
316     ) -> Optional[str]:
317         if not scale_filter:
318             return watermark_filter
319         if not watermark_filter:
320             return scale_filter
321         if watermark_filter.startswith("movie=") and "[in][wm]" in watermark_filter:
322             return (
323                 f"[in]{scale_filter}[scaled];"
324                 + watermark_filter.replace("[in][wm]", "[scaled][wm]", 1)
325             )
326         return f"{scale_filter},{watermark_filter}"
327
328     def _select_encoder(self) -> Tuple[List[str], str]:
329         """Resolve the ``-c:v ?`` args used for EVERY per-clip trim re-encode this run,
330         plus the
331         chosen encoder name. Chosen ONCE (not per clip) so all clips share one codec and
332         the final
333         concat can stay ``-c copy``. NVENC (hardware) is the L4 win on the 4K re-encode;
334         if the
335         requested NVENC encoder can't run in this ffmpeg build (CI / non-GPU box), fall
336         back to
337         libx264 so a misconfig never nukes the reel."""
338         enc = self.encoder
339         if enc in ("h264_nvenc", "hevc_nvenc") and not _probe_encoder_available(enc):
340             logger.warning(
341                 "[compile] encoder %r is not runnable in this ffmpeg build ? falling
342                 back to "
343                 "libx264 (CPU). On the L4 worker this should be available.",
344                 enc,
345             )
346         enc = "libx264"
347         if enc == "libx264":
348             return [
349                 "-c:v",
350                 "libx264",
351                 "-preset",
352                 str(self.encode_profile["video_preset"]),
353                 "-crf",

```

```

348         str(self.encode_profile["crf"]),
349     ], enc
350     # NVENC (H.264/HEVC): hardware encode. -cq is the CRF-equivalent constant-
quality knob; p4 is
351     # a balanced speed/quality preset. yuv420p keeps the output broadly playable.
352     cq = "23"
353     if int(self.encode_profile.get("max_width") or 0) > 0 and int(
354         self.encode_profile.get("max_height") or 0
355     ) > 0:
356         cq = str(self.encode_profile["crf"])
357     pix_fmt = str(self.encode_profile.get("pix_fmt") or "yuv420p")
358     return (
359         ["c:v", enc, "-preset", "p4", "-cq", cq, "-pix_fmt", pix_fmt],
360         enc,
361     )
362
363 def _trim_one_segment(
364     self,
365     idx: int,
366     raw_start: Union[int, float, str],
367     raw_end: Union[int, float, str],
368     segments: List[Tuple[float, float]],
369     video_duration: float,
370     tmp_dir: str,
371     raw_video_path: str,
372     video_filter: Optional[str],
373     watermark_fallback_filter: Optional[str],
374     video_codec_args: Optional[List[str]] = None,
375 ) -> Tuple[Optional[str], Optional[Tuple[int, float, float, str]]]:
376     """Trim a single segment into its own clip file (extracted verbatim from the
377     per-segment body of compile_highlights). Returns (clip_path, skip_record): on
378     success the clip path and None; on a skip (invalid/out-of-range/ffmpeg failure)
379     None and the (idx, start, end, reason) record to record in skipped_segments."""
380     # Normalize timestamps to float seconds ? handles float, int,
381     # plain numeric strings, and colon-separated formats like "MM:SS"
382     start = self._parse_time(raw_start)
383     end = self._parse_time(raw_end)
384     segment_label = (
385         f"Segment #{idx + 1}/{len(segments)} [{start:.2f}s - {end:.2f}s]"
386     )
387
388     # Validate the segment times
389     if start < 0:
390         logger.warning(
391             f"{segment_label}: start_time is negative ({start:.2f}s). Clamping to
0."
392         )
393         start = 0.0
394
395     if start >= end:
396         logger.error(
397             f"{segment_label}: start_time ({start:.2f}s) >= end_time ({end:.2f}s).
Skipping invalid segment."
398         )
399         return None, (idx, start, end, "start >= end")
400
401     # Check if segment extends beyond video duration
402     if video_duration > 0:
403         if start >= video_duration:
404             logger.error(
405                 f"{segment_label}: start_time ({start:.2f}s) is beyond the video
duration ({video_duration:.2f}s). "
406                 f"This segment cannot be extracted ? the video ran out. Skipping."
407             )
408             return None, (idx, start, end, "start beyond video end")

```



```

409
410         if end > video_duration:
411             original_end = end
412             end = video_duration
413             logger.warning(
414                 f"{segment_label}: end_time ({original_end:.2f}s) exceeds video
duration ({video_duration:.2f}s). "
415                 f"Clamping end_time to {video_duration:.2f}s. The segment may be
truncated."
416             )
417
418         clip_path = os.path.join(tmp_dir, f"clip_{idx}.mp4")
419
420         # Precise sub-second trim using fast re-encoding to preserve stream headers
421         padded_start = max(0.0, start - self.begin_padding)
422         padded_end = end + self.end_padding
423
424         # Clamp padded_end to video duration if known
425         if video_duration > 0 and padded_end > video_duration:
426             padded_end = video_duration
427             logger.info(
428                 f"{segment_label}: Padded end ({end + self.end_padding:.2f}s) exceeds
video duration. "
429                 f"Clamping to {video_duration:.2f}s (end padding partially applied)."
430             )
431
432         trim_duration = padded_end - padded_start
433         logger.info(
434             f"Trimming {segment_label} -> [{padded_start:.3f}s - {padded_end:.3f}s]
(duration: {trim_duration:.2f}s)"
435         )
436
437         # Default to libx264 (the historical settings) when a caller invokes
_trim_one_segment
438         # directly without resolving an encoder ? keeps this method's old behaviour
intact.
439         codec_args = (
440             video_codec_args
441             if video_codec_args is not None
442             else ["-c:v", "libx264", "-preset", "superfast", "-crf", "22"]
443         )
444         base_cmd = [
445             ffmpeg_bin(),
446             "-y",
447             "-ss",
448             str(round(padded_start, 3)),
449             "-to",
450             str(round(padded_end, 3)),
451             "-i",
452             raw_video_path,
453             *codec_args,
454             "-c:a",
455             "aac",
456             "-b:a",
457             str(self.encode_profile["audio_bitrate"]),
458         ]
459
460         # Try with the watermark first; if that encode fails (e.g. a bad font/logo
461         # path), retry once WITHOUT it so a branding misconfig can never nuke the
462         # whole reel. If a scale filter is active (SD rendition), the fallback keeps
463         # that scale filter so SD cannot silently become an original-size reel.
464         # Clips stay codec-identical either way, so concat -c copy holds.
465         if video_filter:
466             attempts: List[Tuple[Optional[str], bool]] = [(video_filter, False)]
467             if watermark_fallback_filter != video_filter:

```

```

468         attempts.append((watermark_fallback_filter, True))
469     else:
470         attempts = [(None, False)]
471     produced = False
472     last_exc: Optional[subprocess.CalledProcessError] = None
473     cmd = base_cmd + [clip_path]
474     timeout = ffmpeg_timeout_sec()
475     for attempt_filter, without_watermark in attempts:
476         attempt_vf = ["-vf", attempt_filter] if attempt_filter else []
477         cmd = base_cmd + attempt_vf + [clip_path]
478         try:
479             subprocess.run(
480                 cmd, capture_output=True, check=True, timeout=timeout
481             )
482         except subprocess.TimeoutExpired:
483             reason = f"ffmpeg timeout after {timeout_label(timeout)}"
484             logger.error(f"{segment_label}: FFmpeg trim timed out.")
485             print(f"  Command: {' '.join(cmd)}")
486             print(f"    {reason}")
487             return None, (idx, start, end, reason)
488         except subprocess.CalledProcessError as e:
489             last_exc = e
490             if not without_watermark and len(attempts) > 1:
491                 _err = (
492                     e.stderr.decode(errors="replace").strip()[-300:]
493                     if getattr(e, "stderr", None)
494                     else ""
495                 )
496                 logger.warning(
497                     f"{segment_label}: watermark encode failed; retrying without it."
498                     f"ffmpeg: {_err or '(no stderr captured)'}"
499                 )
500                 continue
501                 break
502             if os.path.exists(clip_path) and os.path.getsize(clip_path) > 0:
503                 produced = True
504                 if without_watermark and len(attempts) > 1:
505                     logger.warning(
506                         f"{segment_label}: encoded WITHOUT the watermark (the watermark
507 filter failed)."
508                     )
509                     break
510                 last_exc = None # ffmpeg succeeded but produced nothing
511                 break
512     if produced:
513         return clip_path, None
514     elif last_exc is not None:
515         stderr_msg = last_exc.stderr.decode(errors="replace").strip()
516         logger.error(f"{segment_label}: FFmpeg trim failed.")
517         print(f"  Command: {' '.join(cmd)}")
518         print(
519             f"    FFmpeg stderr: {stderr_msg[-500:]}")
520         ) # Last 500 chars to avoid flooding
521         return None, (idx, start, end, f"ffmpeg error: {stderr_msg[-200:]}")
522     else:
523         logger.error(
524             f"{segment_label}: FFmpeg exited successfully but output clip is missing
525 or empty. Skipping."
526         )
527         return None, (idx, start, end, "empty output clip")
528
529 def compile_highlights(
530     self,

```

```

530         raw_video_path: str,
531         segments: List[Tuple[float, float]],
532         output_filename: str,
533     ) -> str:
534         """
535         Extracts selected segments from a raw video and stitches them together
536         into a seamless highlights file, applying end_padding to prevent abrupt endings.
537
538         .. warning::
539             This method uses a two-stage codec contract: per-clip re-encodes must use
540             IDENTICAL encoder settings (codec, preset, CRF, pixel format) so the final
541             concat can stream-copy (`-c copy`) without re-encoding. Changing the
542             clip encode settings without updating the concat assumptions will produce
543             broken output.
544         """
545         if not segments:
546             raise ValueError("No highlight segments provided to compile.")
547
548         # Never stitch the same footage twice: collapse overlapping/duplicate ranges
549         # (e.g. accumulated re-run segments or chunk-overlap dupes) before trimming.
550         original_count = len(segments)
551         segments = self._merge_overlapping(segments)
552         if len(segments) < original_count:
553             logger.info(
554                 "Merged %d overlapping/duplicate segment(s) before stitching: %d ->
555                 %d.",
556                 original_count - len(segments),
557                 original_count,
558                 len(segments),
559             )
560         if not segments:
561             raise ValueError("No valid (start < end) highlight segments after merge.")
562
563         if not os.path.exists(raw_video_path):
564             raise FileNotFoundError(
565                 f"[COMPILER ERROR] Source video file not found: {raw_video_path}"
566             )
567
568         # Defense-in-depth: never let the output name escape output_dir (the API also
569         # validates this at the request boundary).
570         output_path = os.path.join(self.output_dir, os.path.basename(output_filename))
571
572         # Probe video duration so we can detect out-of-range segments
573         video_duration = self._get_video_duration(raw_video_path)
574         if video_duration > 0:
575             logger.info(f"Source video duration: {video_duration:.2f}s")
576         else:
577             logger.warning(
578                 "Video duration unknown. Cannot validate segment boundaries against
579                 video length."
580             )
581
582         # We will create temporary clips and stitch them using FFmpeg concat demuxer
583         temp_clips = []
584         skipped_segments = []
585
586         # Create a temp directory within output_dir
587         with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
588             logger.info(
589                 f"Trimming {len(segments)} segments
590                 (begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)..."
591             )
592             # Build the (optional) watermark filtergraph once; it is applied during
593             # per-clip re-encode so the mark is baked into each clip identically.

```

```

591         scale_filter = self._scale_filter()
592         watermark_filter = self._watermark_filter(tmp_dir)
593         video_filter = self._combine_video_filters(scale_filter, watermark_filter)
594         watermark_fallback_filter = scale_filter if watermark_filter else
video_filter
595         if scale_filter:
596             logger.info("[rendition] applying video filter: %s", scale_filter)
597         if watermark_filter:
598             logger.info("[watermark] applying branding overlay to each clip.")
599         # Resolve the video encoder ONCE (probe NVENC / fall back) so every clip
shares one codec
600         # and the final concat stays -c copy (#521).
601         video_codec_args, chosen_encoder = self._select_encoder()
602         if chosen_encoder != "libx264":
603             logger.info("[compile] encoding clips with %s (hardware).",
chosen_encoder)
604         for idx, (raw_start, raw_end) in enumerate(segments):
605             clip_path, skip_record = self._trim_one_segment(
606                 idx,
607                 raw_start,
608                 raw_end,
609                 segments,
610                 video_duration,
611                 tmp_dir,
612                 raw_video_path,
613                 video_filter,
614                 watermark_fallback_filter,
615                 video_codec_args,
616             )
617             if clip_path is not None:
618                 temp_clips.append(clip_path)
619             if skip_record is not None:
620                 skipped_segments.append(skip_record)
621
622         # Report summary of skipped segments
623         if skipped_segments:
624             logger.info(
625                 f"\n[SUMMARY] {len(skipped_segments)} out of {len(segments)}
segments were SKIPPED:"
626             )
627             for seg_idx, seg_start, seg_end, reason in skipped_segments:
628                 logger.info(
629                     f" - Segment #{seg_idx + 1} [{seg_start:.2f}s -
{seg_end:.2f}s]: {reason}"
630                 )
631             logger.info("")
632
633         if not temp_clips:
634             raise RuntimeError(
635                 f"[COMPILER ERROR] Failed to generate ANY valid highlight clips. "
636                 f"All {len(segments)} segments were skipped. "
637                 f"Check the errors above for details. "
638                 f"Common causes: video file is corrupted, segments are out of range,
or ffmpeg is not installed."
639             )
640
641         logger.info(
642             f"Successfully trimmed {len(temp_clips)}/{len(segments)} clips.
Preparing to stitch..."
643         )
644
645         # Create a list file for FFmpeg concat demuxer
646         list_file_path = os.path.join(tmp_dir, "clips.txt")
647         # encoding pinned: a non-ASCII clip path would crash on Windows (cp1252
default);

```

```

648         # ffmpeg's concat demuxer reads the list as UTF-8 on every OS.
649         with open(list_file_path, "w", encoding="utf-8") as f:
650             for clip in temp_clips:
651                 # The concat demuxer resolves relative entries against the list
652                 # directory. Use absolute paths so a temp-dir clip does not become
653                 # output/tmp.../output/tmp.../clip_0.mp4 on Windows.
654                 safe_path = Path(clip).resolve().as_posix().replace("'", "'\\'")
655                 f.write(f"file '{safe_path}'\n")
656
657         logger.info("Stitching clips together...")
658         # Stitch them together without re-encoding (since they were encoded with
659         identical settings)
660         concat_cmd = [
661             ffmpeg_bin(),
662             "-y",
663             "-f",
664             "concat",
665             "-safe",
666             "0",
667             "-i",
668             list_file_path,
669             "-c",
670             "copy",
671             output_path,
672         ]
673         try:
674             timeout = ffmpeg_timeout_sec()
675             subprocess.run(
676                 concat_cmd, capture_output=True, check=True, timeout=timeout
677             )
678
679             # Final verification
680             if not os.path.exists(output_path) or os.path.getsize(output_path) == 0:
681                 raise RuntimeError(
682                     f"[COMPILER ERROR] FFmpeg concatenation appeared to succeed, "
683                     f"but output file is missing or empty: {output_path}"
684                 )
685
686             output_size_mb = os.path.getsize(output_path) / (1024 * 1024)
687             logger.info(
688                 f"Dynamic highlight reel saved: {output_path} ({output_size_mb:.1f}
689 MB)"
690             )
691             return output_path
692         except subprocess.CalledProcessError as e:
693             stderr_msg = e.stderr.decode(errors="replace").strip()
694             raise RuntimeError(
695                 f"[COMPILER ERROR] FFmpeg concatenation failed.\n"
696                 f"  Command: {' '.join(concat_cmd)}\n"
697                 f"  FFmpeg stderr: {stderr_msg[-500:]}"
698             ) from e
699         except subprocess.TimeoutExpired as e:
700             raise RuntimeError(
701                 f"[COMPILER ERROR] FFmpeg concatenation timed out after
702 {timeout_label(timeout)}.\n"
703                 f"  Command: {' '.join(concat_cmd)}"
704             ) from e

```

3. user (2026-07-08T06:16:37.231Z)

dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine placement (#521) (#525)

```
bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells
($9) (#522)
699d85b feat(validation): cache validation verdict by video_id to skip re-validation on reupload
(#518)
71c88ef docs(runlogs): validation-latency closure report for #512 + #513 (cloud-serving) (#519)
115b83d docs(runlogs): record live smoke of the crash-before-marker fast-fail (#515) (#517)
ef41473 fix(cloud-run): fail fast when a worker execution terminates without a done-marker
(#515)
f5c9dd9 chore(ui): re-vendor SPA bundle with the hosted compute-picker cloud default (#516)
ec2d080 fix(serving): fail fast when compute=local forces weightless in-process detection (#511)
```

4. assistant (2026-07-08T06:17:30.592Z)

Let me verify where the NVENC encoder is actually configured/reachable, to gauge whether the failure scenario is realistically triggerable.

5. user (2026-07-08T06:17:32.721Z)

```
backend\config\models.py:37:          "recordings, and other encoders may use different GOP
sizes ? tune "
backend\config\models.py:388:      # Video encoder for the per-clip trim re-encode (the branding
watermark forces a re-encode; the
backend\config\models.py:389:      # concat stays -c copy). Default ``libx264`` (CPU, works
everywhere incl. CI). ``h264_nvenc`` /
backend\config\models.py:390:      # ``hevc_nvenc`` are the L4 HARDWARE encoders ? dramatically
faster on the 4K re-encode that
backend\config\models.py:391:      # dominated the 2026-07-08 CUJ stitch (#521). The compiler
probes the requested encoder ONCE and
backend\config\models.py:393:      # cloud compile-dispatch forwards
``deployment.cloud_stitch_encoder`` onto this for the L4 run;
backend\config\models.py:395:      encoder: Literal["libx264", "h264_nvenc", "hevc_nvenc"] =
"libx264"
backend\config\models.py:536:      # Video encoder the cloud compile-dispatch forwards onto
``stitching.encoder`` for the L4 stitch
backend\config\models.py:537:      # (#521). ``h264_nvenc`` uses the L4's hardware encoder for the
4K re-encode the watermark forces;
backend\config\models.py:539:      # the control-plane's own local stitch always uses
``stitching.encoder`` (libx264 by default).
backend\config\models.py:540:      cloud_stitch_encoder: Literal["libx264", "h264_nvenc",
"hevc_nvenc"] = "h264_nvenc"
tests\test_compile_dispatch.py:132:def
test_compile_dispatch_stages_resolved_spec_with_nvenc(compile_env):
tests\test_compile_dispatch.py:134:      and the L4 encoder (deployment.cloud_stitch_encoder =
h264_nvenc by default) ? so the worker
tests\test_compile_dispatch.py:148:      # the L4 stitch uses NVENC (forwarded from
deployment.cloud_stitch_encoder)
tests\test_compile_dispatch.py:149:      assert fake.spec["stitching"]["encoder"] == "h264_nvenc"
tests\test_config_deployment.py:73:      # #521: per-phase placement (unset ? legacy default)
+ the L4 stitch encoder.
tests\test_config_deployment.py:75:      "cloud_stitch_encoder": "h264_nvenc",
tests\test_config_deployment.py:404:      "cloud_stitch_encoder": "h264_nvenc",
backend\orchestration.py:1217:      with ``stitching.encoder`` (NVENC on the L4, libx264 on the
CP), fetches the source, runs the
backend\orchestration.py:1244:      encoder=getattr(stitching, "encoder", "libx264"),
backend\orchestration.py:1366:      # the stitching config is the control-plane's RESOLVED one,
with the encoder swapped to the L4's
backend\orchestration.py:1367:      # (deployment.cloud_stitch_encoder, NVENC by default) so the
worker hardware-encodes the reel.
backend\orchestration.py:1369:      stitch_dump["encoder"] = getattr(
backend\orchestration.py:1370:      getattr(cfg, "deployment", None), "cloud_stitch_encoder",
"h264_nvenc"
tests\test_compiler.py:227:# -----
encoder selection (#521 ? NVENC)
tests\test_compiler.py:230:def test_select_encoder_default_is_libx264(tmp_path):
tests\test_compiler.py:231:      """"Default encoder = libx264 (CPU) with the historical settings ?
```

```

no probe, works everywhere."""
tests\test_compiler.py:233:     args, chosen = comp._select_encoder()
tests\test_compiler.py:238: def test_select_encoder_nvenc_when_available(tmp_path, monkeypatch):
tests\test_compiler.py:239:     """h264_nvenc resolves to the hardware encoder args when the
probe reports it runnable (the L4)."""
tests\test_compiler.py:242:     monkeypatch.setattr(comp_mod, "_probe_encoder_available", lambda
enc: True)
tests\test_compiler.py:243:     comp = VideoCompiler(str(tmp_path / "out"), encoder="h264_nvenc")
tests\test_compiler.py:244:     args, chosen = comp._select_encoder()
tests\test_compiler.py:245:     assert chosen == "h264_nvenc"
tests\test_compiler.py:246:     assert args[2] == ["-c:v", "h264_nvenc"]
tests\test_compiler.py:250: def
test_select_encoder_falls_back_to_libx264_when_nvenc_unavailable(tmp_path, monkeypatch):
tests\test_compiler.py:251:     """On a box where NVENC can't actually run (CI / no GPU), the
requested encoder falls back to
tests\test_compiler.py:255:     monkeypatch.setattr(comp_mod, "_probe_encoder_available", lambda
enc: False)
tests\test_compiler.py:256:     comp = VideoCompiler(str(tmp_path / "out"), encoder="h264_nvenc")
tests\test_compiler.py:257:     args, chosen = comp._select_encoder()
tests\test_compiler.py:262: def test_compile_uses_selected_encoder_in_trim(tmp_path,
monkeypatch):
tests\test_compiler.py:263:     """End-to-end (ffmpeg mocked): with an available NVENC encoder,
every per-clip trim command
tests\test_compiler.py:264:     carries -c:v h264_nvenc, and the concat stays -c copy."""
tests\test_compiler.py:267:     monkeypatch.setattr(comp_mod, "_probe_encoder_available", lambda
enc: True)
tests\test_compiler.py:268:     comp = VideoCompiler(str(tmp_path / "out"), encoder="h264_nvenc")
tests\test_compiler.py:287:     assert "h264_nvenc" in cmd # hardware encode on every clip
backend\pipeline\stitcher\compiler.py:26:#: Per-encoder availability memo (#521): probing ffmpeg
is a subprocess, so a run's first clip pays
backend\pipeline\stitcher\compiler.py:27:#: it and the rest reuse the verdict. Keyed by encoder
name; cleared only across processes.
backend\pipeline\stitcher\compiler.py:31: def _probe_encoder_available(encoder: str) -> bool:
backend\pipeline\stitcher\compiler.py:32:     """Can this ffmpeg build actually RUN ``encoder``
(not merely list it)? Runs a tiny 1-frame
backend\pipeline\stitcher\compiler.py:34:     real test encode ? not ``-encoders`` grep ? is what
distinguishes "usable" from "will fail at
backend\pipeline\stitcher\compiler.py:36:     if encoder in _ENCODER_AVAILABLE:
backend\pipeline\stitcher\compiler.py:37:         return _ENCODER_AVAILABLE[encoder]
backend\pipeline\stitcher\compiler.py:40:     # budget, or 30 s when unbounded) so a wedged
encoder can't hang the whole stitch.
backend\pipeline\stitcher\compiler.py:55:         encoder,
backend\pipeline\stitcher\compiler.py:66:         _ENCODER_AVAILABLE[encoder] = ok
backend\pipeline\stitcher\compiler.py:77:         encoder: str = "libx264",
# Video encoder for the per-clip trim re-encode
backend\pipeline\stitcher\compiler.py:86:         # works everywhere; "h264_nvenc"/"hevc_nvenc"
(#521). "libx264" (CPU) is the default and
backend\pipeline\stitcher\compiler.py:87:         # re-encode the watermark forces. The requested
use the L4's HARDWARE encoder for the 4K
backend\pipeline\stitcher\compiler.py:88:         # (_select_encoder) and falls back to libx264
encoder is PROBED once per run
backend\pipeline\stitcher\compiler.py:89:         # self.encoder = (encoder or "libx264").strip()
if this ffmpeg build can't run it.
backend\pipeline\stitcher\compiler.py:90:         or "libx264"
backend\pipeline\stitcher\compiler.py:327:     def _select_encoder(self) -> Tuple[List[str],
str]:
backend\pipeline\stitcher\compiler.py:329:         chosen encoder name. Chosen ONCE (not per
clip) so all clips share one codec and the final
backend\pipeline\stitcher\compiler.py:331:         requested NVENC encoder can't run in this
ffmpeg build (CI / non-GPU box), fall back to
backend\pipeline\stitcher\compiler.py:333:         enc = self.encoder
backend\pipeline\stitcher\compiler.py:334:         if enc in ("h264_nvenc", "hevc_nvenc") and not
_probe_encoder_available(enc):
backend\pipeline\stitcher\compiler.py:336:             "[compile] encoder %r is not runnable
in this ffmpeg build ? falling back to "

```

```

backend\pipeline\stitcher\compiler.py:438:      # directly without resolving an encoder ?
keeps this method's old behaviour intact.
backend\pipeline\stitcher\compiler.py:540:      IDENTICAL encoder settings (codec, preset,
CRF, pixel format) so the final
backend\pipeline\stitcher\compiler.py:599:      # Resolve the video encoder ONCE (probe
NVENC / fall back) so every clip shares one codec
backend\pipeline\stitcher\compiler.py:601:      video_codec_args, chosen_encoder =
self._select_encoder()
backend\pipeline\stitcher\compiler.py:602:      if chosen_encoder != "libx264":
backend\pipeline\stitcher\compiler.py:603:          logger.info("[compile] encoding clips
with %s (hardware).", chosen_encoder)
backend\worker\_main_.py:503:      f"encoder={stitching.encoder} rendition={rendition}
out={out_name}"
backend\compute\base.py:67:      # Raw worker ``--set k=v`` overrides (storage.output_root/backend
+ stitching.encoder). A tuple
backend\compute\cloud_run.py:333:      # never re-dispatches, so the only overrides are the
CP-forwarded storage/encoder knobs.
tests\test_worker.py:973:      bucket, video_id, source, name, encoder="h264_nvenc",
rendition="original"
tests\test_worker.py:982:      "stitching": {"encoder": encoder, "watermark": {"enabled":
False}},
tests\test_worker.py:1017:      seen["encoder"] = stitching.encoder

```

[Showing results with pagination = limit: 80]

6. user (2026-07-08T06:17:33.544Z)

```

1      """Single resolver for the ffmpeg / ffprobe binaries (PACKAGING_PLAN.md P2a).
2
3      Every ffmpeg/ffprobe invocation in the engine goes through :func:`ffmpeg_bin` /
:func:`ffprobe_bin`
4      so a packaged app can point at a vendored or non-PATH binary via the ``RALLY_FFMPEG`` /
5      ``RALLY_FFPROBE`` env vars, there is ONE place to add a config knob or a louder check
later, and a
6      missing ffmpeg fails with an actionable, OS-specific install hint instead of a cryptic
7      ``FileNotFoundError`` from deep in the pipeline.
8
9      DEFAULT (env unset) returns the bare name (``"ffmpeg"`` / ``"ffprobe"``) ? PATH
resolution, exactly
10     the prior hardcoded literals (byte-identical argv), so this is a pure seam with no
behaviour change.
11     stdlib-only.
12     """
13
14     from __future__ import annotations
15
16     import logging
17     import os
18     import platform
19     import shutil
20     from typing import Optional
21
22     logger = logging.getLogger(__name__)
23
24     DEFAULT_FFPROBE_TIMEOUT_SEC = 60.0
25     DEFAULT_FFMPEG_TIMEOUT_SEC = 1800.0
26
27
28     class FfmpegNotFound(RuntimeError):
29         """Raised by :func:`require_ffmpeg` when ffmpeg/ffprobe can't be located."""
30
31
32     def ffmpeg_bin() -> str:
33         """The ffmpeg executable: ``RALLY_FFMPEG`` if set, else ``"ffmpeg"`` (resolved on
PATH)."""

```



```

34         return (os.environ.get("RALLY_FFMPEG") or "").strip() or "ffmpeg"
35
36
37     def ffprobe_bin() -> str:
38         """The ffprobe executable: ``RALLY_FFPROBE`` if set, else ``"ffprobe"`` (resolved on
PATH)."""
39         return (os.environ.get("RALLY_FFPROBE") or "").strip() or "ffprobe"
40
41
42     def _timeout_seconds(env_var: str, default: float) -> Optional[float]:
43         raw = os.environ.get(env_var)
44         if raw is None or raw.strip() == "":
45             value = default
46         else:
47             try:
48                 value = float(raw)
49             except ValueError:
50                 logger.warning("%s=%r is not numeric; using %.0fs", env_var, raw, default)
51                 value = default
52         if value <= 0:
53             return None
54         return value
55
56
57     def ffprobe_timeout_sec() -> Optional[float]:
58         """Bound metadata probes. Set ``RALLY_FFPROBE_TIMEOUT_SEC=0`` to wait forever."""
59         return _timeout_seconds("RALLY_FFPROBE_TIMEOUT_SEC", DEFAULT_FFPROBE_TIMEOUT_SEC)
60
61
62     def ffmpeg_timeout_sec() -> Optional[float]:
63         """Bound transcode/extract/concat calls. Set ``RALLY_FFMPEG_TIMEOUT_SEC=0`` to wait
forever."""
64         return _timeout_seconds("RALLY_FFMPEG_TIMEOUT_SEC", DEFAULT_FFMPEG_TIMEOUT_SEC)
65
66
67     def timeout_label(seconds: Optional[float]) -> str:
68         return "forever" if seconds is None else f"{seconds:g}s"
69
70
71     def ffmpeg_available() -> bool:
72         return shutil.which(ffmpeg_bin()) is not None
73
74
75     def ffprobe_available() -> bool:
76         return shutil.which(ffprobe_bin()) is not None
77
78
79     def install_hint() -> str:
80         """The OS-appropriate one-line install command (mirrors scripts/doctor.py)."""
81         system = platform.system()
82         if system == "Windows":
83             return "winget install Gyan.FFmpeg"
84         if system == "Darwin":
85             return "brew install ffmpeg"
86         return "sudo apt-get install -y ffmpeg (or your distro's package manager)"
87
88
89     def require_ffmpeg() -> None:
90         """Raise :class:`FfmpegNotFound` (with an OS-specific install hint) if
ffmpeg/ffprobe is missing.
91
92         A loud, actionable pre-flight for entry points that are about to shell out ? used by
the
93         first-run self-test / wizard; the per-call-site argv still goes through
:func:`ffmpeg_bin`."""

```

```

94         missing = [
95             name
96             for name, ok in (
97                 ("ffmpeg", ffmpeg_available()),
98                 ("ffprobe", ffprobe_available()),
99             )
100             if not ok
101         ]
102     if missing:
103         raise FfmpegNotFound(
104             f"'{name}' and '{path}' not found (PATH or RALLY_FFMPEG/RALLY_FFPROBE).
Ffmpeg is "
105             f"required for video processing. Install it: {install_hint()}"
106         )
107

```

7. assistant (2026-07-08T06:18:08.632Z)

I have enough to evaluate the claim. Let me trace the reachability once more against the config defaults.

The mechanism the reviewer describes is technically coherent:

- `\_probe\_encoder\_available` (compiler.py:44-62) test-encodes with only `-c:v <encoder>` and no preset/quality flags.
- `\_select\_encoder` (compiler.py:358-361) returns `["-c:v", enc, "-preset", "p4", "-cq", cq, "-pix\_fmt", pix\_fmt]` for the real per-clip encode.
- If `-preset p4` were rejected while bare NVENC init succeeds, every `\_trim\_one\_segment` would fail (the without-watermark retry reuses the same `codec\_args`, so it fails too), `temp\_clips` empties, and `compile\_highlights` raises RuntimeError (compiler.py:633-639). The failure is committed once for the whole run, so there is no per-clip fallback.

But the failure is not reachable in any supported/default configuration:

1. Default: `stitching.encoder = "libx264"` (models.py:395). libx264 takes the no-probe branch (compiler.py:341-349) ? never touches the NVENC path.
2. CI / non-GPU: NVENC init itself fails, so `\_probe\_encoder\_available` returns False and `\_select\_encoder` falls back to libx264 (compiler.py:334-340). No p4 encode ever runs.
3. The only place NVENC is selected is the L4 Cloud Run stitch worker (`deployment.cloud\_stitch\_encoder = "h264\_nvenc"`, models.py:540, forwarded in orchestration.py:1369-1370) ? a controlled, modern ffmpeg image where the p1?p7 preset scheme is supported.

The `-preset p4` naming has existed in ffmpeg since v4.3 (mid-2020). The scenario requires a real NVIDIA GPU present AND an ffmpeg predating that AND someone deliberately setting the non-default `h264\_nvenc` ? a combination that is neither the default nor the deployment target. The reviewer itself concedes "low confidence" and "modern ffmpeg on the L4 image is unaffected." A probe is inherently a best-effort heuristic (it also differs in input resolution, pixel format, level limits); the one flag it omits only bites on a ~6-year-old ffmpeg that does not exist in this project's GCP-only, L4-served deployment.

Per the instruction to default to false when the failure cannot be concretely confirmed in a reachable configuration, I refute.

8. user (2026-07-08T06:18:16.569Z)

Structured output provided successfully